There are two ways to tell Razor that a value is a literal string and not C# code. The first way is to prefix the first instance of a literal string on each line with `@:`.

Listing 3.7 Rendering literal strings in Razor

```
@foreach (var city in cities)
{
    if (city.Country == "UK")
    {
        @:Country:  @city.Country, Name: @city.Name
    }
}
```

Alternatively, you can use the `<text>` tag to surround the content. This works particularly well if you are working with multiline content, as shown in the following listing.

Listing 3.8 Using the text tag to render multiline literal strings

```
@foreach (var city in cities)
{
    if (city.Country == "UK")
    {
        <text>Country:  @city.Country<br />
        Name: @city.Name</text>
    }
}
```

The text tags are not rendered as part of the output; only their content is rendered. In addition, using text tags results in whitespace being removed from the output that appears before or after them.

3.1.5 Rendering literal HTML

The default behavior of Razor is to render any markup within the Razor page literally but to apply HTML encoding to the result of all expressions that resolve to strings. Any non-ASCII characters, as well as potentially *unsafe* characters (those that could facilitate the injection of malicious scripts into a web page), such as `<`, `>`, `&`, and `“`, are encoded to their HTML equivalents: `<`, `>`, `&`,